Classicmodels API demo (Tasks 1–6)

Run **all cells from top to bottom** with the API server running.

Prerequisites

- MySQL `classicmodels` loaded; `.env` configured (copy from `.env.example`).
- From `W4111-Template_Web_Application`: `python -m app.main` or `uvicorn app.main:app --reload --port 8000`.
- `pip install httpx` (and Jupyter if needed).

TEST CUSTOMER cells create `W4111_NOTEBOOK_TEST`, then delete it. A final cell deletes any leftover row with that name if a run stopped mid-way.

In [1]:
```
import os

import httpx

BASE = os.environ.get("API_BASE_URL", "http://127.0.0.1:8000").rstrip("/")

# Filled by the POST cell; kept through DELETE so we can assert 404 on GET.
NOTEBOOK_TEST_CUSTOMER_ID: int | None = None

with httpx.Client(base_url=BASE, timeout=30.0) as client:
    r = client.get("/health")
    r.raise_for_status()
    print("health:", r.json())
```

```
health: {'status': 'ok'}
```

Customers — `GET /customers`

In [2]:
```
with httpx.Client(base_url=BASE, timeout=30.0) as client:
    r = client.get("/customers", params={"country": "France"})
    r.raise_for_status()
    body = r.json()
    items = body.get("items", [])
    print("count (France):", len(items))
    if items:
        print("sample:", items[0])
```

```
count (France): 12
sample: {'customerNumber': 103, 'customerName': 'Atelier graphique', 'contac
tLastName': 'Schmitt', 'contactFirstName': 'Carine ', 'phone': '40.32.2555',
'addressLine1': '54, rue Royale', 'addressLine2': None, 'city': 'Nantes', 's
tate': None, 'postalCode': '44000', 'country': 'France', 'salesRepEmployeeNu
mber': 1370, 'creditLimit': 21000.0}
```

TEST CUSTOMER — `POST /customers`

Body uses **`customerName: W4111_NOTEBOOK_TEST`** plus required columns.

In [3]:
```
TEST_CUSTOMER_BODY = {
    "customerName": "W4111_NOTEBOOK_TEST",
    "contactLastName": "Notebook",
    "contactFirstName": "Test",
    "phone": "555-0100",
    "addressLine1": "1 Scholar Lane",
    "city": "New York",
    "country": "USA",
}

with httpx.Client(base_url=BASE, timeout=30.0) as client:
    r = client.post("/customers", json=TEST_CUSTOMER_BODY)
    r.raise_for_status()
    NOTEBOOK_TEST_CUSTOMER_ID = int(r.json())
    print("POST /customers -> customerNumber:", NOTEBOOK_TEST_CUSTOMER_ID)
```

```
POST /customers -> customerNumber: 497
```

TEST CUSTOMER — `GET /customers/{customerNumber}`

In [4]:
```
assert NOTEBOOK_TEST_CUSTOMER_ID is not None

with httpx.Client(base_url=BASE, timeout=30.0) as client:
    r = client.get(f"/customers/{NOTEBOOK_TEST_CUSTOMER_ID}")
    r.raise_for_status()
    print(r.json())
```

```
{'customerNumber': 497, 'customerName': 'W4111_NOTEBOOK_TEST', 'contactLastN
ame': 'Notebook', 'contactFirstName': 'Test', 'phone': '555-0100', 'addressL
ine1': '1 Scholar Lane', 'addressLine2': None, 'city': 'New York', 'state':
None, 'postalCode': None, 'country': 'USA', 'salesRepEmployeeNumber': None,
'creditLimit': None}
```

TEST CUSTOMER — `PUT /customers/{customerNumber}`

In [5]:
```
assert NOTEBOOK_TEST_CUSTOMER_ID is not None

with httpx.Client(base_url=BASE, timeout=30.0) as client:
    r = client.put(
        f"/customers/{NOTEBOOK_TEST_CUSTOMER_ID}",
        json={"phone": "555-0199"},
    )
    r.raise_for_status()
```

```
    print("PUT:", r.json())

    r2 = client.get(f"/customers/{NOTEBOOK_TEST_CUSTOMER_ID}")
    r2.raise_for_status()
    print("phone after PUT:", r2.json()["phone"])
```

```
PUT: {'updated': 1}
phone after PUT: 555-0199
```

TEST CUSTOMER — `DELETE /customers/{customerNumber}`

In [6]:
```
assert NOTEBOOK_TEST_CUSTOMER_ID is not None

with httpx.Client(base_url=BASE, timeout=30.0) as client:
    r = client.delete(f"/customers/{NOTEBOOK_TEST_CUSTOMER_ID}")
    r.raise_for_status()
    print("DELETE:", r.json())
```

```
DELETE: {'deleted': 1}
```

TEST CUSTOMER — `GET` after delete (404)

In [7]:
```
assert NOTEBOOK_TEST_CUSTOMER_ID is not None

with httpx.Client(base_url=BASE, timeout=30.0) as client:
    r = client.get(f"/customers/{NOTEBOOK_TEST_CUSTOMER_ID}")
    print("status:", r.status_code)
    print("body:", r.json())
    assert r.status_code == 404

NOTEBOOK_TEST_CUSTOMER_ID = None
```

```
status: 404
body: {'detail': "No customer with customerNumber '497'"}
```

Read-only — orders and order details

In [8]:
```
SAMPLE_ORDER_NUMBER: int | None = None
SAMPLE_PRODUCT_CODE: str | None = None

with httpx.Client(base_url=BASE, timeout=30.0) as client:
    r = client.get("/orders", params={"status": "Shipped"})
    r.raise_for_status()
    orders = r.json().get("items", [])
    print("GET /orders?status=Shipped — count:", len(orders))
    if not orders:
        raise RuntimeError("No shipped orders in DB; pick another filter for
    SAMPLE_ORDER_NUMBER = int(orders[0]["orderNumber"])
    print("sample orderNumber:", SAMPLE_ORDER_NUMBER)
```

```
    r1 = client.get(f"/orders/{SAMPLE_ORDER_NUMBER}")
    r1.raise_for_status()
    print("GET /orders/{orderNumber}:", r1.json())

    r2 = client.get("/orderdetails", params={"orderNumber": SAMPLE_ORDER_NUM
    r2.raise_for_status()
    lines = r2.json().get("items", [])
    print("GET /orderdetails?orderNumber=... — lines:", len(lines))

    r3 = client.get(f"/orders/{SAMPLE_ORDER_NUMBER}/orderdetails")
    r3.raise_for_status()
    scoped = r3.json().get("items", [])
    print("GET /orders/{orderNumber}/orderdetails — lines:", len(scoped))
    assert len(scoped) == len(lines)

    if scoped:
        SAMPLE_PRODUCT_CODE = str(scoped[0]["productCode"])
        r4 = client.get(
            f"/orders/{SAMPLE_ORDER_NUMBER}/orderdetails/{SAMPLE_PRODUCT_COD
        )
        r4.raise_for_status()
        print("GET composite row:", r4.json())
```

```
GET /orders?status=Shipped — count: 303
sample orderNumber: 10100
GET /orders/{orderNumber}: {'orderNumber': 10100, 'orderDate': '2003-01-06',
'requiredDate': '2003-01-13', 'shippedDate': '2003-01-10', 'status': 'Shippe
d', 'comments': None, 'customerNumber': 363}
GET /orderdetails?orderNumber=... — lines: 4
GET /orders/{orderNumber}/orderdetails — lines: 4
GET composite row: {'orderNumber': 10100, 'productCode': 'S18_1749', 'quanti
tyOrdered': 32, 'priceEach': 136.0, 'orderLineNumber': 3}
```

Cleanup — remove stray `W4111_NOTEBOOK_TEST` rows

If a previous run stopped before **DELETE**, delete by name so reruns stay idempotent.

In [9]:
```
with httpx.Client(base_url=BASE, timeout=30.0) as client:
    r = client.get(
        "/customers",
        params={"customerName": "W4111_NOTEBOOK_TEST"},
    )
    r.raise_for_status()
    for row in r.json().get("items", []):
        cid = row["customerNumber"]
        d = client.delete(f"/customers/{cid}")
        print(f"cleanup DELETE /customers/{cid} ->", d.status_code, d.json()
print("cleanup done")
```

```
cleanup done
```